

PENJELASAN SOURCE CODE

Reli Gita Nurhidayati (2311102025)

1. main.dart

File **main.dart** adalah file utama yang pertama kali dijalankan saat aplikasi dibuka. Di sinilah seluruh alur aplikasi dimulai, mulai dari inialisasi, pengaturan tampilan, pengelolaan state, hingga komunikasi dengan Android native untuk notifikasi.

Fungsi main()

Fungsi `main()` adalah entry point aplikasi Dart. Isinya hanya satu baris, yaitu `runApp(const MyApp())`, yang berfungsi untuk memuat dan menjalankan widget utama. Tidak ada inialisasi tambahan karena notifikasi ditangani langsung oleh native Android, bukan melalui plugin Flutter.

Class MyApp

Class **MyApp** merupakan `StatelessWidget` yang berperan sebagai root dari seluruh widget tree. Di dalamnya digunakan `MaterialApp` untuk mendefinisikan tema aplikasi. Warna utama yang digunakan adalah ungu (`0xFF6C63FF`) dengan latar belakang ungu muda (`0xFFF3F1FF`). Banner debug juga dinonaktifkan agar tampilan lebih bersih, dan `HomePage` ditetapkan sebagai halaman pertama yang ditampilkan.

Class HomePage & State Management

Halaman utama dibuat menggunakan **StatefulWidget** karena ada satu data yang bisa berubah di tengah jalannya aplikasi, yaitu foto yang dipilih pengguna. Data tersebut disimpan dalam variabel `_image` bertipe `File?` (nullable, artinya bisa kosong). Setiap kali nilai `_image` berubah, `setState()` dipanggil agar Flutter merender ulang tampilan dan foto baru langsung terlihat di layar.

Fungsi _getImage()

Fungsi ini menerima satu parameter bertipe `ImageSource`, yang menentukan apakah gambar diambil dari kamera atau galeri. Secara internal, fungsi memanggil `_picker.pickImage(source: source)` dan menunggu hasilnya secara asinkron menggunakan `await`. Kalau pengguna memilih gambar (tidak membatalkan), file

disimpan ke `_image` lalu fungsi `_showNotification()` dipanggil. Kalau pengguna membatalkan, tidak ada yang berubah.

Fungsi `_showNotification()`

Fungsi ini menggunakan **MethodChannel** dengan nama `'com.example.notifications'` untuk mengirim perintah ke Android native. Perintah yang dikirim adalah method `'showNotification'`. Proses ini dibungkus dengan `try-catch` untuk menangani kemungkinan error, misalnya jika channel tidak tersedia di platform tertentu.

Method `build()`

Method `build()` berisi seluruh susunan tampilan halaman. Strukturnya adalah Scaffold dengan AppBar berwarna ungu di atas, dan body berupa Column yang menyusun tiga elemen secara vertikal: teks identitas mahasiswa di bagian atas, kotak penampil foto di tengah, dan dua tombol di bawah. Kotak foto menggunakan logika kondisional: jika `_image == null` maka tampilkan placeholder, jika tidak tampilkan `Image.file(_image!)` dengan sudut yang sudah dibulatkan menggunakan `ClipRRect`.

2. MainActivity.kt

File **MainActivity.kt** adalah activity utama Android yang juga berperan sebagai penerima perintah dari Flutter melalui Platform Channel. File ini ditulis menggunakan Kotlin dan berfungsi sebagai jembatan antara logika Flutter dan API notifikasi Android.

`configureFlutterEngine()`

Method **`configureFlutterEngine()`** di-override dari class induk `FlutterActivity`. Di sinilah `MethodChannel` didaftarkan dengan nama yang sama persis seperti yang digunakan di sisi Flutter, yaitu `'com.example.notifications'`. Ketika Flutter mengirim perintah melalui channel ini, Android akan memeriksa nama method-nya. Jika nama method-nya `'showNotification'`, maka fungsi `showNotification()` dijalankan dan `result.success(null)` dikembalikan ke Flutter sebagai tanda bahwa perintah berhasil dieksekusi.

showNotification()

Fungsi **showNotification()** berisi logika lengkap untuk menampilkan notifikasi sistem Android. Langkah pertama adalah membuat `NotificationChannel` jika versi Android adalah 8.0 (API 26) atau lebih baru. Channel ini wajib ada karena Android 8.0 ke atas mengharuskan setiap notifikasi terdaftar dalam sebuah channel dengan ID, nama, dan tingkat kepentingan yang jelas.

Setelah channel siap, notifikasi dibangun menggunakan `NotificationCompat.Builder`. Icon yang digunakan adalah `ic_menu_camera` dari resource bawaan Android, judulnya 'Foto Berhasil Dimuat!', dan isi pesannya menyebut nama mahasiswa. Prioritas notifikasi diset ke `PRIORITY_HIGH` agar notifikasi langsung muncul sebagai heads-up di bagian atas layar. Terakhir, `notificationManager.notify(1, notification)` dipanggil untuk menampilkan notifikasi tersebut ke pengguna.

3. AndroidManifest.xml

File **AndroidManifest.xml** adalah file konfigurasi wajib di setiap proyek Android. Di sinilah kita mendaftarkan semua permission yang dibutuhkan aplikasi agar sistem Android mengizinkan akses ke fitur tertentu. Jika permission tidak didaftarkan, aplikasi bisa langsung crash atau fiturnya tidak berjalan.

Berikut permission yang ditambahkan dan alasannya:

Permission	Alasan Dibutuhkan
POST_NOTIFICATIONS	Wajib untuk Android 13 ke atas. Tanpa ini, notifikasi tidak akan muncul sama sekali meskipun kode sudah benar.
VIBRATE	Diperlukan agar perangkat bisa bergetar saat notifikasi muncul, menambah responsivitas aplikasi.
CAMERA	Dibutuhkan agar aplikasi bisa membuka kamera bawaan perangkat ketika tombol Kamera ditekan.
READ_EXTERNAL_STORAGE	Untuk membaca file gambar dari penyimpanan eksternal pada perangkat Android versi lama (di bawah Android 13).
READ_MEDIA_IMAGES	Pengganti <code>READ_EXTERNAL_STORAGE</code> untuk Android 13 ke atas, khusus untuk mengakses file gambar dari galeri.

Selain permission, file ini juga mendefinisikan `MainActivity` sebagai activity utama dengan intent-filter `android.intent.action.MAIN` dan category `LAUNCHER`. Konfigurasi ini yang membuat ikon aplikasi muncul di launcher perangkat dan bisa diklik untuk membuka aplikasi.